

JavaScript Complete Course

Beginner to Advanced — Simple Definitions & Code Examples

A self-paced reference guide covering every core JavaScript topic

Module 1: Introduction

What is JavaScript?

Definition: JavaScript is a programming language that adds behavior and interactivity to websites. HTML is the structure, CSS is the style, JavaScript is the behavior.

```
console.log("Hello, JavaScript!");
```

Output: Hello, JavaScript!

History of JavaScript

Definition: JavaScript was created by Brendan Eich in 1995 in just 10 days for Netscape. It was first called Mocha, then LiveScript, then JavaScript. It has nothing to do with Java — the name was a marketing choice.

ECMAScript

Definition: ECMAScript (ES) is the official standard/rulebook that JavaScript follows. Different versions (ES5, ES6/ES2015, ES2020...) added new features over time.

```
// ES6 (2015) example – arrow function, a modern feature
const add = (a, b) => a + b;
console.log(add(2, 3));
```

Output: 5

Browser vs Node.js

Definition: JavaScript can run in two places: the **browser** (Chrome, Firefox — for websites) and **Node.js** (a runtime that lets JS run outside the browser, e.g. on a server).

Browser JS	Node.js
Has access to DOM, window	Has access to file system, servers
Runs on client's machine	Runs on a server/your computer

JavaScript Engine

Definition: A JavaScript engine is a program that reads and executes JS code. Chrome uses **V8**, Firefox uses **SpiderMonkey**. Node.js also uses V8.

How JavaScript Works

Definition: The engine parses your code, converts it to machine-readable form, and executes it line by line inside an execution context (explained more in Module 15).

Internal vs External JS

Definition: Internal JS is written directly inside an HTML file using a script tag. External JS is written in a separate .js file and linked to HTML.

```
<!-- Internal -->
<script>
  console.log("Internal JS");
</script>
```

```
<!-- External -->
<script src="app.js"></script>
```

Linking JS Files

Definition: You link a JS file to HTML using the src attribute. Placing the script tag just before the closing body tag ensures the HTML loads first.

```
<body>
  ...
  <script src="script.js"></script>
</body>
```

Console

Definition: The console is a panel in browser dev tools where JavaScript output, errors, and logs appear. Open it with F12 or right-click then Inspect.

```
console.log("normal message");
console.error("error message");
console.warn("warning message");
```

Comments

Definition: Comments are notes in code that JavaScript ignores. They explain code to humans.

```
// This is a single-line comment

/* This is a
   multi-line comment */
console.log("Comments don't run"); // this runs fine
```

Module 2: Variables and Data Types

Variables

Definition: A variable is a named container that stores a value in memory, so you can use or change it later.

```
let city = "Hyderabad";
console.log(city);
```

Output: Hyderabad

var

Definition: var is the oldest way to declare a variable. It is function-scoped and can be re-declared. Avoid using it in modern code.

```
var age = 25;
var age = 26; // allowed, can cause bugs
console.log(age);
```

Output: 26

let

Definition: let declares a variable that can be reassigned but not re-declared in the same scope. It is block-scoped.

```
let score = 10;
score = 20; // allowed
console.log(score);
```

Output: 20

const

Definition: const declares a variable whose value cannot be reassigned after creation. Use it by default unless you know the value will change.

```
const pi = 3.14;
console.log(pi);
// pi = 3.15; // Error: Assignment to constant variable
```

Naming Rules

Definition: Variable names can contain letters, numbers, \$ and _. They cannot start with a number and cannot be reserved words like let or if.

```
let studentName = "Ravi"; // valid, camelCase
let _age = 25;           // valid
let 1name = "x";         // invalid – starts with a number
```

Primitive Data Types

Definition: Primitive types hold a single simple value: String, Number, Boolean, Null, Undefined, BigInt, Symbol.

```
let name = "Asha"; // String
let age = 22;      // Number
let isStudent = true; // Boolean
let car = null;    // Null
let city;          // Undefined
```

Non-Primitive Data Types

Definition: Non-primitive (reference) types can hold collections of values: Object, Array, Function.

```
let student = { name: "Asha", age: 22 }; // Object
let marks = [90, 85, 70];               // Array
function greet() { console.log("Hi"); }  // Function
```

typeof

Definition: typeof is an operator that tells you the data type of a value.

```
console.log(typeof "hello"); // string
console.log(typeof 25);      // number
console.log(typeof true);    // boolean
console.log(typeof undefined); // undefined
```

null

Definition: null means "intentionally empty" — a value you set yourself to represent nothing.

```
let selectedCity = null;
console.log(selectedCity); // null
```

undefined

Definition: undefined means a variable has been declared but not yet given a value. JavaScript sets this automatically.

```
let x;  
console.log(x); // undefined
```

BigInt

Definition: BigInt represents numbers larger than the normal Number type can safely handle. You create one by adding "n" at the end.

```
let big = 123456789012345678901234567890n;  
console.log(big);
```

Symbol

Definition: Symbol creates a completely unique value, often used as a hidden/unique object property key.

```
let id1 = Symbol("id");  
let id2 = Symbol("id");  
console.log(id1 === id2); // false – always unique
```

Module 3: Operators

Arithmetic Operators

Definition: Used to do math: + - * / % (remainder) ** (power).

```
console.log(10 + 3); // 13  
console.log(10 % 3); // 1  
console.log(2 ** 3); // 8
```

Assignment Operators

Definition: Used to assign or update a variable's value: = += -= *= /=

```
let total = 10;  
total += 5; // same as total = total + 5  
console.log(total); // 15
```

Comparison Operators

Definition: Compare two values and return true/false: == != === !== > < >= <= . Use === and !== (strict) to also check data type.

```
console.log(5 == "5"); // true (loose, ignores type)  
console.log(5 === "5"); // false (strict, checks type too)
```

Logical Operators

Definition: Combine boolean expressions: && (AND), || (OR), ! (NOT).

```
let age = 20;  
console.log(age > 18 && age < 60); // true  
console.log(age < 18 || age > 60); // false
```

Increment / Decrement

Definition: ++ increases a number by 1, -- decreases it by 1.

```
let count = 5;
count++;
console.log(count); // 6
```

String Operators

Definition: The + operator joins (concatenates) strings together.

```
let first = "Hello";
let second = "World";
console.log(first + " " + second); // Hello World
```

Ternary Operator

Definition: A shortcut for if-else in one line: condition ? valueIfTrue : valueIfFalse.

```
let age = 20;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result); // Adult
```

Nullish Coalescing (??)

Definition: Returns the right value only if the left is null or undefined (not for other falsy values like 0 or "").

```
let username = null;
console.log(username ?? "Guest"); // Guest
```

Optional Chaining (?.)

Definition: Safely accesses a nested object property without crashing if a parent is missing.

```
let user = { name: "Ravi" };
console.log(user.address?.city); // undefined, no error
```

Module 4: Type Conversion

Implicit Conversion

Definition: JavaScript automatically converts one type to another when needed.

```
console.log("5" + 1); // "51" (number becomes string)
console.log("5" - 1); // 4 (string becomes number)
```

Explicit Conversion

Definition: You manually convert a value from one type to another using functions like Number(), String(), Boolean().

```
let str = "10";
let num = Number(str);
console.log(num + 5); // 15
```

Number()

Definition: Converts a value into a Number.

```
console.log(Number("42")); // 42
console.log(Number("abc")); // NaN
```

String()

Definition: Converts a value into a String.

```
console.log(String(100)); // "100"
```

Boolean()

Definition: Converts a value into true or false. 0, "", null, undefined, NaN become false; everything else becomes true.

```
console.log(Boolean(0)); // false
console.log(Boolean("hi")); // true
```

parseInt()

Definition: Converts a string into a whole number, reading digits until a non-digit appears.

```
console.log(parseInt("25px")); // 25
```

parseFloat()

Definition: Converts a string into a decimal number.

```
console.log(parseFloat("3.14kg")); // 3.14
```

Module 5: Control Statements

if / else / else if

Definition: Runs a block of code only if a condition is true; else/else if handle other cases.

```
let marks = 75;
if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

Output: Grade B

switch

Definition: Compares a value against multiple possible cases, useful as an alternative to many else-if blocks.

```
let day = 2;
switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
  default: console.log("Other day");
}
```

Output: Tuesday

Nested if

Definition: An if statement placed inside another if statement, for checking multiple conditions in sequence.

```
let age = 20, hasID = true;
if (age >= 18) {
  if (hasID) {
    console.log("Entry allowed");
  }
}
```

Ternary (as control flow)

Definition: Used as a compact if-else for simple decisions and assignments.

```
let num = 7;
console.log(num % 2 === 0 ? "Even" : "Odd"); // Odd
```


Module 6: Loops

for loop

Definition: Repeats a block of code a specific number of times.

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Output: 1 2 3 4 5 (each on new line)

while loop

Definition: Repeats a block of code as long as a condition remains true. Condition is checked before each run.

```
let i = 1;  
while (i <= 3) {  
  console.log(i);  
  i++;  
}
```

do while loop

Definition: Like while, but runs the code block at least once before checking the condition.

```
let i = 1;  
do {  
  console.log(i);  
  i++;  
} while (i <= 3);
```

for...of

Definition: Loops through the values of an iterable, like an array or string.

```
let fruits = ["apple", "mango", "banana"];  
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

for...in

Definition: Loops through the keys (property names) of an object.

```
let student = { name: "Ravi", age: 22 };  
for (let key in student) {  
  console.log(key, student[key]);  
}
```

break

Definition: Immediately exits a loop, stopping it completely.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) break;  
  console.log(i);  
}
```

```
}  
// Output: 1 2
```

continue

Definition: Skips the current loop iteration and moves to the next one.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;  
  console.log(i);  
}  
// Output: 1 2 4 5
```

Module 7: Functions

Function Declaration

Definition: A reusable block of code defined with the function keyword and a name. It is hoisted (can be called before it's defined).

```
function greet(name) {  
  return "Hello " + name;  
}  
console.log(greet("Asha")); // Hello Asha
```

Function Expression

Definition: A function stored inside a variable. It is not hoisted like a declaration.

```
const greet = function(name) {  
  return "Hi " + name;  
};  
console.log(greet("Ravi")); // Hi Ravi
```

Arrow Functions

Definition: A shorter syntax for writing functions, introduced in ES6. Does not have its own "this".

```
const add = (a, b) => a + b;  
console.log(add(4, 5)); // 9
```

Parameters & Arguments

Definition: Parameters are placeholders in a function definition; arguments are the actual values passed when calling it.

```
function multiply(a, b) { // a, b = parameters  
  return a * b;  
}  
console.log(multiply(3, 4)); // 3, 4 = arguments -> 12
```

Return

Definition: The return keyword sends a value back from the function and stops it from running further.

```
function square(n) {  
  return n * n;  
}  
console.log(square(5)); // 25
```

Callback Functions

Definition: A function passed as an argument into another function, to be run later.

```
function processOrder(callback) {
  console.log("Order placed");
  callback();
}
processOrder(function() {
  console.log("Callback: Order confirmed");
});
```

Higher-Order Functions

Definition: A function that takes another function as an argument or returns a function.

```
function greetMaker(greeting) {
  return function(name) {
    console.log(greeting + " " + name);
  };
}
const sayHi = greetMaker("Hi");
sayHi("Asha"); // Hi Asha
```

Scope

Definition: Scope defines where a variable can be accessed. Global scope = everywhere; local/block scope = only inside its function or block.

```
let globalVar = "I am global";
function show() {
  let localVar = "I am local";
  console.log(globalVar); // accessible
}
show();
// console.log(localVar); // Error: not accessible outside
```

Hoisting

Definition: JavaScript moves variable and function declarations to the top of their scope before running the code. var and function declarations are hoisted; let/const are not usable before declaration.

```
console.log(a); // undefined, not an error
var a = 5;
```

Closures

Definition: A closure is a function that "remembers" variables from its outer scope even after the outer function has finished running.

```
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}
const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
```

Module 8: Strings

String Methods

Definition: Built-in functions to read or transform text.

```
let name = "Hyderabad";
console.log(name.length);           // 9
console.log(name.toUpperCase());    // HYDERABAD
console.log(name.toLowerCase());    // hyderabad
console.log(name.slice(0, 4));      // Hyde
console.log(name.includes("bad"));  // true
console.log(name.trim());           // removes extra spaces
console.log(name.replace("Hyder", "Alla")); // Allaabad
```

Template Literals

Definition: Strings written with backticks (` ) that allow embedding variables directly using `${ }`.

```
let name = "Ravi";
let age = 22;
console.log(`My name is ${name} and I am ${age} years old.`);
```

Output: My name is Ravi and I am 22 years old.

Escape Characters

Definition: Special character combinations starting with a backslash used inside strings, like `\n` (new line) and `\"` (quote).

```
console.log("Line1\nLine2");
console.log("She said \"Hello\"");
```

Module 9: Arrays

Creating & Accessing Arrays

Definition: An array is an ordered list of values, accessed using index numbers starting from 0.

```
let fruits = ["apple", "mango", "banana"];
console.log(fruits[0]); // apple
fruits[1] = "grape"; // updating
console.log(fruits);
```

Looping Arrays

```
let nums = [1, 2, 3];
for (let n of nums) {
  console.log(n);
}
```

Array Methods

Definition: Built-in functions to add, remove, transform, or search array items.

```
let arr = [1, 2, 3];
arr.push(4);      // [1,2,3,4] add to end
arr.pop();        // [1,2,3] remove from end
arr.unshift(0);   // [0,1,2,3] add to start
arr.shift();      // [1,2,3] remove from start
console.log(arr.slice(0,2)); // [1,2] copy portion, doesn't change original
console.log(arr.splice(1,1)); // removes 1 item at index 1, changes original
console.log(arr.concat([9,9])); // joins arrays
console.log(arr.join("-"));    // "1-2-3" -> string
console.log(arr.includes(2));  // true
console.log(arr.indexOf(3));   // position of value

console.log(arr.map(n => n * 2)); // new array, each item doubled
console.log(arr.filter(n => n > 1)); // new array, only matching items
console.log(arr.reduce((sum, n) => sum + n, 0)); // single combined value
console.log(arr.find(n => n > 1)); // first matching item
console.log(arr.some(n => n > 2)); // true if any item matches
console.log(arr.every(n => n > 0)); // true if all items match
console.log(arr.sort()); // sorts array
console.log(arr.reverse()); // reverses array
arr.forEach(n => console.log(n)); // runs a function for each item
```

map/filter/reduce/find are the most commonly used in real projects – practice these first.

Module 10: Objects

Creating Objects, Properties & Methods

Definition: An object stores data as key-value pairs. A function stored inside an object is called a method.

```
let student = {
  name: "Asha",
  age: 22,
  greet: function() {
    console.log("Hi, I am " + this.name);
  }
};
console.log(student.name); // Asha (property)
```

```
student.greet(); // Hi, I am Asha (method)
```

Object.keys() / values() / entries()

Definition: These get an object's keys, values, or key-value pairs as arrays.

```
let student = { name: "Asha", age: 22 };
console.log(Object.keys(student)); // ["name", "age"]
console.log(Object.values(student)); // ["Asha", 22]
console.log(Object.entries(student)); // [["name", "Asha"], ["age", 22]]
```

Destructuring

Definition: A shortcut to pull values out of an object or array into separate variables.

```
let student = { name: "Asha", age: 22 };
let { name, age } = student;
console.log(name, age); // Asha 22

let [a, b] = [10, 20];
console.log(a, b); // 10 20
```

Spread Operator (...)

Definition: Expands an array or object into individual elements — often used to copy or merge them.

```
let arr1 = [1, 2];
let arr2 = [...arr1, 3, 4];
console.log(arr2); // [1,2,3,4]

let obj1 = { a: 1 };
let obj2 = { ...obj1, b: 2 };
console.log(obj2); // {a:1, b:2}
```

Rest Operator (...)

Definition: Collects multiple values into a single array — the opposite use-case of spread, typically used in function parameters.

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
```

Module 11: Date and Math

Date Object

Definition: Used to work with dates and times.

```
let now = new Date();
console.log(now.getFullYear()); // e.g. 2026
console.log(now.getMonth());    // 0-11 (0 = January)
console.log(now.getDate());     // day of month
console.log(now.getDay());      // 0-6 (0 = Sunday)
```

Math Object

Definition: A built-in object with math-related methods and constants.

```
console.log(Math.round(4.6)); // 5
console.log(Math.floor(4.9)); // 4
console.log(Math.ceil(4.1));  // 5
console.log(Math.max(1, 5, 3)); // 5
console.log(Math.min(1, 5, 3)); // 1
console.log(Math.random());    // random number between 0 and 1
console.log(Math.PI);          // 3.14159...
```

Module 12: DOM Manipulation

DOM (Document Object Model)

Definition: The DOM is a tree-like structure that represents your HTML page in memory, letting JavaScript read and change it.

Selecting Elements

```
document.getElementById("title");
document.querySelector(".card"); // first match
document.querySelectorAll(".card"); // all matches (list)
```

Changing Text & HTML

```
let title = document.querySelector("#title");
title.textContent = "New Text"; // safely changes text
title.innerHTML = "<b>Bold Text</b>"; // changes HTML content
```

Changing CSS

```
let box = document.querySelector(".box");
box.style.color = "blue";
box.style.backgroundColor = "yellow";
box.classList.add("active");
```

Attributes

```
let img = document.querySelector("img");
img.setAttribute("src", "photo.jpg");
```

```
console.log(img.getAttribute("src"));
```

Creating & Removing Elements

```
let newDiv = document.createElement("div");
newDiv.textContent = "I'm new!";
document.body.appendChild(newDiv); // add to page
newDiv.remove(); // remove from page
```

Forms

```
let input = document.querySelector("#username");
console.log(input.value); // reads what user typed
```

Events & Event Listeners

Definition: An event is an action (click, type, hover). An event listener is code that runs when that event happens.

```
let btn = document.querySelector("#submitBtn");
btn.addEventListener("click", function() {
  console.log("Button was clicked!");
});
```

Event Delegation

Definition: Attaching one event listener to a parent element instead of many children, using event.target to detect which child was clicked.

```
document.querySelector("#list").addEventListener("click", function(e) {
  if (e.target.tagName === "LI") {
    console.log("Clicked item:", e.target.textContent);
  }
});
```

Module 13: ES6+ Features Recap

Most ES6+ features are covered in earlier modules. Here is a quick summary table:

Feature	Purpose
let / const	Block-scoped variables (Module 2)
Arrow functions	Shorter function syntax (Module 7)
Template literals	Embed variables in strings (Module 8)
Destructuring	Extract values from objects/arrays (Module 10)
Spread / Rest	Expand or collect values (Module 10)

Default Parameters

Definition: Lets a function parameter use a fallback value if no argument is passed.

```
function greet(name = "Guest") {
  console.log("Hello " + name);
}
greet(); // Hello Guest
greet("Ravi"); // Hello Ravi
```


Modules (import/export)

Definition: Modules let you split code across files and share code between them using export and import.

```
// math.js
export function add(a, b) { return a + b; }

// app.js
import { add } from "./math.js";
console.log(add(2, 3)); // 5
```

Classes

Definition: A class is a blueprint for creating objects with shared properties and methods. Covered in detail in Module 18.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log("Hi, I'm " + this.name);
  }
}
let p = new Person("Ravi");
p.greet(); // Hi, I'm Ravi
```

Module 14: Asynchronous JavaScript

Call Stack

Definition: The call stack is where JavaScript keeps track of function calls, running them one at a time, last-in-first-out.

Web APIs

Definition: Browser-provided features (like `setTimeout`, `fetch`, DOM events) that JavaScript can use, even though they aren't part of the core language.

Event Loop

Definition: The event loop constantly checks if the call stack is empty, then moves waiting tasks (like a finished `setTimeout`) into the stack to run — this is how JS handles async without freezing.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");
// Output: 1 3 2 (setTimeout waits for the stack to clear)
```

Callback

Definition: A function passed to run after an async task completes.

```
setTimeout(function() {
  console.log("Runs after 2 seconds");
}, 2000);
```

Promise

Definition: A Promise represents a value that will be available later — either resolved (success) or rejected (failure).

```
let promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) resolve("Data loaded");
  else reject("Error occurred");
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

Async/Await

Definition: A cleaner way to work with Promises. `async` marks a function as asynchronous; `await` pauses it until a Promise resolves.

```
async function loadData() {
  console.log("Loading...");
  let result = await new Promise(resolve => {
    setTimeout(() => resolve("Data ready"), 1000);
  });
  console.log(result);
}
loadData();
```

Fetch API

Definition: A built-in function to make HTTP requests to servers/APIs and get data back.

```
async function getUser() {
  let response = await fetch("https://api.example.com/user/1");
  let data = await response.json();
  console.log(data);
}
```

JSON

Definition: JSON (JavaScript Object Notation) is a text format for exchanging data. `JSON.stringify()` converts JS to JSON text; `JSON.parse()` converts JSON text back to JS.

```
let obj = { name: "Ravi", age: 22 };
let jsonStr = JSON.stringify(obj);
console.log(jsonStr); // '{"name":"Ravi","age":22}'
console.log(JSON.parse(jsonStr).name); // Ravi
```

Error Handling (try/catch)

Definition: try/catch lets you run code and "catch" any errors instead of crashing the program.

```
try {
  let result = riskyFunction();
} catch (error) {
  console.log("Something went wrong:", error.message);
} finally {
  console.log("This always runs");
}
```

Module 15: Advanced JavaScript

Execution Context & Memory

Definition: An execution context is the environment where JS code runs — it has a Memory (Variable) phase where variables/functions are set up, and an Execution phase where code runs line by line.

this Keyword

Definition: "this" refers to the object that is currently calling the function. Its value depends on how the function is called.

```
const student = {
  name: "Asha",
  greet() {
    console.log(this.name); // "this" = student object
  }
};
student.greet(); // Asha
```

call, apply, bind

Definition: Methods to control what "this" refers to inside a function. `call/apply` run the function immediately; `bind` returns a new function for later use.

```
function greet() { console.log("Hi " + this.name); }
let person = { name: "Ravi" };
greet.call(person);           // Hi Ravi
greet.apply(person);          // Hi Ravi
let boundGreet = greet.bind(person);
boundGreet();                  // Hi Ravi
```

Prototype & Prototype Chain

Definition: Every object has a hidden link to another object called its prototype, from which it can inherit properties/methods. This chain of links is the prototype chain.

```
let animal = { eats: true };
let dog = Object.create(animal);
console.log(dog.eats); // true - inherited via prototype chain
```

Closures (recap)

Definition: Already covered in Module 7 — a function that remembers its outer scope's variables.

Hoisting (recap)

Definition: Already covered in Module 7 — declarations are moved to the top of scope before execution.

Event Bubbling

Definition: When an event happens on an element, it "bubbles up" and also triggers on its parent elements.

```
document.querySelector("#child").addEventListener("click", () => console.log("Child clicked"));
document.querySelector("#parent").addEventListener("click", () => console.log("Parent clicked"));
// Clicking child logs: "Child clicked" then "Parent clicked"
```

Event Capturing

Definition: The opposite of bubbling — the event travels from the outer parent down to the target element first. Enabled by passing true as a third argument.

```
element.addEventListener("click", handler, true); // capturing phase
```

Debouncing

Definition: Delays running a function until the user stops triggering it for a set time — useful for search boxes.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
let search = debounce(() => console.log("Searching..."), 500);
```

Throttling

Definition: Ensures a function runs at most once every fixed interval — useful for scroll events.

```
function throttle(fn, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      fn(...args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```


Module 16: Browser Storage

Cookies

Definition: Small pieces of data stored in the browser, sent to the server with every request. Have an expiry date.

```
document.cookie = "username=Ravi; expires=Fri, 31 Dec 2026 23:59:59 GMT";
```

Local Storage

Definition: Stores data in the browser with no expiry date — persists even after closing the browser.

```
localStorage.setItem("username", "Ravi");
console.log(localStorage.getItem("username")); // Ravi
localStorage.removeItem("username");
```

Session Storage

Definition: Like localStorage, but data is cleared when the browser tab is closed.

```
sessionStorage.setItem("token", "abc123");
console.log(sessionStorage.getItem("token"));
```

Module 17: APIs

REST API

Definition: A REST API is a set of rules that lets applications communicate over HTTP using standard methods, usually exchanging data in JSON format.

HTTP Methods

Method	Purpose
GET	Read/fetch data
POST	Create new data
PUT	Update/replace existing data
DELETE	Remove data

CRUD

Definition: Create, Read, Update, Delete — the four basic operations of working with data, mapped to POST, GET, PUT, DELETE.

Fetch & API Integration

```
async function createUser(name) {
  let response = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: name })
  });
  let data = await response.json();
  console.log(data);
}
```


Module 18: Object-Oriented Programming

Classes & Objects

Definition: A class is a blueprint; an object is a specific instance created from that blueprint.

```
class Car {
  constructor(brand) {
    this.brand = brand;
  }
}
let myCar = new Car("Toyota"); // object created from class
console.log(myCar.brand); // Toyota
```

Constructor

Definition: A special method that runs automatically when a new object is created from a class, used to set initial values.

```
class Student {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}
let s1 = new Student("Asha", 22);
console.log(s1.name, s1.age); // Asha 22
```

Inheritance

Definition: Allows one class (child) to reuse properties and methods from another class (parent) using extends.

```
class Animal {
  speak() { console.log("Animal makes a sound"); }
}
class Dog extends Animal {
  bark() { console.log("Dog barks"); }
}
let d = new Dog();
d.speak(); // inherited
d.bark();
```

Encapsulation

Definition: Bundling data and methods together inside a class, and hiding internal details using private fields (#).

```
class Account {
  #balance = 0; // private field
  deposit(amount) {
    this.#balance += amount;
  }
  getBalance() {
    return this.#balance;
  }
}
let acc = new Account();
acc.deposit(500);
console.log(acc.getBalance()); // 500
```

Polymorphism

Definition: Different classes can have a method with the same name but different behavior.

```
class Cat { sound() { console.log("Meow"); } }
class Dog { sound() { console.log("Woof"); } }
[new Cat(), new Dog()].forEach(animal => animal.sound());
// Meow, Woof – same method name, different behavior
```

Abstraction

Definition: Hiding complex implementation details and showing only what's necessary to the user of a class.

```
class CoffeeMachine {
  makeCoffee() {
    this.#grindBeans(); // hidden internal steps
    this.#boilWater();
    console.log("Coffee ready!");
  }
  #grindBeans() { /* internal detail */ }
  #boilWater() { /* internal detail */ }
}
new CoffeeMachine().makeCoffee(); // user only sees this simple method
```

Module 19: Modules

Import / Export

Definition: Modules let you organize code into separate files and share functionality between them.

Named Export

```
// utils.js
export function add(a, b) { return a + b; }
export const PI = 3.14;

// app.js
import { add, PI } from './utils.js';
```

Default Export

```
// greet.js
export default function greet(name) {
  console.log("Hello " + name);
}

// app.js
import greet from './greet.js'; // no curly braces for default export
greet("Ravi");
```

Module 20: Interview Preparation — Key Concepts

Frequently Asked Concepts

Question	Short Answer
var vs let vs const?	var is function-scoped and re-declarable; let is block-scoped and reassignable; const is block-scoped and cannot be reassigned.
== vs ===?	== compares values only (loose); === compares value and type (strict).
What is a closure?	A function that remembers variables from its outer scope even after that scope has finished.
What is hoisting?	Declarations are moved to the top of their scope before code runs.
What is the event loop?	The mechanism that lets JS handle async tasks by moving completed tasks into the call stack once it's empty.
Promise vs async/await?	async/await is syntax built on top of Promises that makes async code look synchronous and easier to read.
What is "this"?	Refers to the object calling the function; its value depends on how the function is invoked.
What is the DOM?	A tree structure representing the HTML page that JS can read and modify.

Output-Based Question Example

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// Output: 1, 4, 3, 2
// Reason: synchronous code runs first, then Promises (microtasks),
// then setTimeout (macrotask)
```

Debugging Tip

Definition: Use `console.log()` at different points, or the browser's Debugger/Breakpoints (Sources tab in DevTools) to pause code and inspect variable values step by step.

Module 21: Mini Projects — Starter Code

Calculator

```
function calculate(a, b, operator) {
  if (operator === "+") return a + b;
  if (operator === "-") return a - b;
  if (operator === "*") return a * b;
  if (operator === "/") return a / b;
}
console.log(calculate(10, 5, "+")); // 15
```

Counter

```
let count = 0;
document.querySelector("#increment").addEventListener("click", () => {
  count++;
  document.querySelector("#count").textContent = count;
});
```

Digital Clock

```
function updateClock() {
  let now = new Date();
  document.querySelector("#clock").textContent = now.toLocaleTimeString();
}
setInterval(updateClock, 1000);
```

To-Do List

```
let tasks = [];
function addTask(task) {
  tasks.push(task);
  console.log("Tasks:", tasks);
}
addTask("Learn JavaScript");
addTask("Build a project");
```

Quiz App (structure)

```
let questions = [
  { q: "2 + 2 = ?", answer: 4 },
  { q: "Capital of India?", answer: "Delhi" }
];
let score = 0;
questions.forEach(item => {
  console.log(item.q);
  // compare user input to item.answer, increase score if correct
});
```

Weather App (structure)

```
async function getWeather(city) {
  let response = await fetch(`https://api.example.com/weather?city=${city}`);
  let data = await response.json();
  console.log(data.temperature);
}
```

```
}
```

Expense Tracker (structure)

```
let expenses = [];  
function addExpense(name, amount) {  
  expenses.push({ name, amount });  
}  
function getTotal() {  
  return expenses.reduce((sum, e) => sum + e.amount, 0);  
}  
addExpense("Food", 200);  
addExpense("Travel", 100);  
console.log(getTotal()); // 300
```

Notes App (structure)

```
let notes = JSON.parse(localStorage.getItem("notes")) || [];  
function addNote(text) {  
  notes.push(text);  
  localStorage.setItem("notes", JSON.stringify(notes));  
}
```

Password Generator

```
function generatePassword(length) {  
  let chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";  
  let password = "";  
  for (let i = 0; i < length; i++) {  
    password += chars[Math.floor(Math.random() * chars.length)];  
  }  
  return password;  
}  
console.log(generatePassword(10));
```

GitHub Profile Finder (structure)

```
async function findUser(username) {  
  let response = await fetch(`https://api.github.com/users/${username}`);  
  let data = await response.json();  
  console.log(data.name, data.public_repos);  
}  
findUser("octocat");
```

You're Ready!

You now have a complete reference covering every core JavaScript topic — from variables to closures, async/await to OOP, and 10 mini-project starters. Next step: practice by rebuilding each mini-project fully, then move on to React.js and Node.js.